Intelligent Phishing Email Analyzer

Automated Assessment and Explanation

Project Detail	Information
Course	CSC482 Capstone Project II
Team	Capstone Project Team 4
Team Leader	Yuuki
Team Members	Joseph and Bivaina
Project Duration	June 1 to July 28

1. Project Overview

The goal of this capstone project is to build a web-based phishing email analysis tool for Security Operation Center analysts. The tool will accept suspicious email content through either .eml file upload or pasted raw email text, analyze the message for phishing indicators, and generate a structured report.

The report will include a phishing probability score, key indicators of compromise, suspicious links, attachment names, sender/header concerns, and a plain-language explanation that can help junior analysts understand why the email is suspicious or likely safe.

2. Problem Statement

Security Operation Centers receive large volumes of suspicious emails. Analysts must quickly decide whether each email is a phishing attempt, but manual review can be slow, inconsistent, and difficult for junior analysts.

Phishing emails may include spoofed senders, suspicious URLs, urgent language, malicious attachments, fake login pages, or impersonation attempts. This project addresses the problem by creating a tool that automates the initial assessment and explains its reasoning in clear language.

3. Project Objectives

1. Build a web-based tool that accepts .eml files and pasted email content.
2. Extract important email components such as headers, sender, subject, body, links, and attachment names.
3. Analyze the email for phishing indicators using prompt engineering and an LLM.
4. Generate a phishing probability score.
5. Identify indicators of compromise such as suspicious links, impersonation, urgency, spoofing, and attachment risks.
6. Produce a readable report for SOC analysts and junior security staff.
7. Provide a final demo showing the full workflow from email input to generated report.

4. Scope

In Scope

- Uploading .eml files.
- Pasting raw email content into the web tool.
- Parsing email headers, body text, links, and attachment names.
- Detecting suspicious patterns such as spoofing, urgency, credential theft attempts, and mismatched URLs.
- Generating a phishing probability score and natural-language explanation.
- Displaying a structured analysis report in the web interface.
- Preparing documentation and a final presentation/demo.

Out of Scope

- Full enterprise SOC integration.
- Real-time email gateway deployment.
- Automatic deletion or quarantine of emails.
- Advanced malware analysis of attachments.
- Guaranteed detection of all phishing attacks.
- Production-grade URL sandboxing unless time allows.

5. Key Features

- Email input through .eml upload or pasted raw email text.
- Email parsing for sender, reply-to, subject, date, headers, body, URLs, and attachment names.
- Phishing analysis for spoofing, suspicious links, urgent language, credential theft, fake login indicators, attachment risk, and impersonation.
- Report generation with phishing score, risk level, summary, IoCs, suspicious URLs, header concerns, recommended action, and explanation.

6. Technology Stack

Area	Planned Technologies
Frontend	HTML, CSS, JavaScript, or a selected frontend framework; upload and paste-input forms; report display interface.
Backend	Python or Node.js backend; Python email package or eml-parser; API endpoint for email submission; cleaning and structuring logic.
AI/LLM Integration	GPT-4o, Claude, or another available LLM; few-shot prompt examples; structured output; guardrails for evidence-based reporting.
Optional Enhancements	URL reputation lookup, URL sandbox integration, report export, analyst history, and sample email

	dataset for testing.

7. System Workflow

8. User opens the web-based tool.
9. User uploads a .eml file or pastes email content.
10. Backend parses the email content.
11. System extracts headers, sender details, body text, URLs, and attachments.
12. Parsed data is sent to the LLM analysis prompt.
13. LLM returns a structured phishing assessment.
14. Backend formats the result into a report.
15. Frontend displays the report to the user.
16. User reviews phishing score, IoCs, explanation, and recommended action.

8. Project Timeline

Period	Focus	Key Tasks	Deliverables
Week 1: Jun 1-Jun 5	Course setup and project selection	Review syllabus; set up lab environment; form team; select topic; define initial use case.	Team formed; project topic selected; initial concept documented.
Week 2: Jun 8-Jun 12	Planning and environment setup	Install tools; create project plan, timeline, and schedule; decide stack; assign responsibilities; create repository/workspace; create high-level design.	Project plan; ready environment; architecture/workflow; task schedule.
Week 3: Jun 15-Jun 19	Basic prototype and email input	Build initial web interface; add .eml upload; add pasted email input; create backend endpoint; begin parser; test simple samples.	Working input interface; basic backend connection; initial email parser.
Week 4: Jun 22-Jun 26	Email parsing and data extraction	Extract sender, subject, body, headers, URLs, and attachments; normalize parsed data; add invalid-input handling; display parsed preview; create test cases.	Reliable parser output; parsed email preview; sample test emails.
Week 5: Jun 29-Jul 3	LLM phishing analysis	Design prompt; define structured output; integrate selected LLM API; test prompt quality; improve vague or unsupported responses.	Working LLM analysis; initial phishing report; prompt documentation.

Week 6: Jul 6-Jul 10	Report generation and UI improvements	Design final report layout; add risk labels and probability score; display IoCs; add recommendations; improve styling, validation, and loading states.	Polished report interface; improved experience; complete end-to-end workflow.
Week 7: Jul 13-Jul 17	Testing, refinement, and documentation	Test phishing and legitimate examples; fix bugs; improve scoring consistency; prepare user guide; begin slides; assign demo roles.	Tested application; documentation draft; presentation draft; demo script.
Week 8: Jul 20-Jul 26	Final polish and presentation preparation	Finalize features; complete documentation; finalize slides; rehearse demo; prepare backup screenshots; confirm presentation sections.	Final web tool; documentation; presentation; demo-ready project.
Final: Jul 27-Jul 28	Presentation and demo	Present problem, solution, architecture, and implementation; demonstrate email input and report; explain results and limitations; answer questions.	Final presentation; live or recorded demo; completed submission.

9. Team Roles and Responsibilities

Person	Primary Role	Responsibilities
Yuuki	Team Leader and Project Coordinator	Coordinate meetings and task assignments; track schedule and deliverables; define scope and requirements; review documentation and presentation; support integration.
Joseph	Backend and Email Parsing	Implement backend endpoints; handle .eml processing; extract headers, body, links, and attachments; add parser error handling; support LLM integration.
Bivaina	Frontend and Report Interface	Build the user interface; create upload and paste-input sections; design report display; improve usability and visual layout; support final presentation visuals.
All Members	Shared Project Work	Test sample emails; review LLM output quality; prepare slides; practice the demo; write and

		review documentation.

10. Risk Management

Risk	Impact	Mitigation
LLM output is vague or inconsistent	High	Use structured prompts and required output fields.
Email parsing fails on complex .eml files	Medium	Start with common formats and add error handling.
API access or cost issues	High	Prepare fallback using mock analysis or local rule-based scoring.
Time constraints	High	Prioritize core upload, parsing, analysis, and report features.
Team workload imbalance	Medium	Use weekly task assignments and progress check-ins.
Demo failure	Medium	Prepare screenshots, sample outputs, and backup demo data.

11. Testing Plan

Testing will cover normal use, invalid input, and report quality. The team will use safe sample phishing and legitimate emails to verify parser behavior and generated explanations.

- Valid .eml file upload and invalid file upload.
- Pasted raw email text.
- Legitimate email examples and phishing email examples.
- Emails with suspicious links, attachments, or missing headers.
- LLM response formatting and required report fields.
- Report display on different screen sizes.

Expected result: The system should consistently produce a readable report with a score, risk level, IoCs, explanation, and recommended next steps.

12. Success Criteria

17. The web tool accepts .eml files and pasted email content.
18. The system extracts key email details.
19. The system generates a phishing probability score.
20. The report identifies meaningful phishing indicators.
21. The explanation is understandable for junior analysts.
22. The team can demonstrate the tool during the final presentation.
23. Documentation clearly explains the system design, usage, and limitations.

13. Final Deliverables

- Web-based phishing email analyzer.
- Source code repository.
- Project plan document.
- Technical documentation or user guide.
- Final presentation slides.
- Demo script or backup screenshots.
- Final project demonstration.

14. Limitations

The tool is intended to assist analysts, not replace them. LLM-generated analysis may contain errors or uncertainty, so final decisions should be reviewed by a human analyst. The system may not detect every phishing attempt, especially highly targeted or novel attacks. URL sandboxing and attachment malware analysis may be limited depending on available time and resources.

15. Future Improvements

- URL reputation checking.
- Attachment scanning integration.
- Analyst login and case history.
- Report export to PDF.
- Integration with ticketing systems or email security platforms.
- Model comparison between GPT-4o, Claude, and local models.
- Dashboard showing phishing trends over time.

Signature